

ASCENT Central Db Liquibase Deployment Process

1. You will need to make separate copies of your liquibase.properties files. Each file will have the database connection values for the target database you want to push changes to. Here's an example of how I created my files:
 - a. branchdev_liquibase.properties: Connection values for my individual developer database
 - b. maindev_liquibase.properties: Connection values for the main development database that we all push our changes to
 - c. prod_liquibase.properties: Connection values for the production database
 - i. Each file will have the following properties updated for the intended target
 1. Liquibase.command.url
 2. Liquibase.command.username
 3. Liquibase.command.password
2. You can test which changes in the changelog have not yet been applied to the target database using the following command
 - a. **>_ liquibase status --defaults-file=maindev_liquibase.properties**
(specify the file that contains the connection values for your intended target)
 - b. **Note:** The following two changesets will always show in the results
pre-post_deploy.xml::1::db.admin
pre-post_deploy.xml::2::db.admin
3. If you'd like, you can run the following command to see the SQL that Liquibase will generate from your changeset before applying:
 - a. **>_ liquibase update-sql --defaults-file=branchdev_liquibase.properties**
4. When you're ready to apply your changeset, execute the following command:
 - a. **liquibase update --defaults-file=branchdev_liquibase.properties**
(note: the update command does not have the sql tag included)

5. When doing database development on your individual developer database, you may encounter a tricky situation on applying your changeset if you've already made the changes against the database before creating the changeset. Liquibase will not know that the changes have been applied already and will attempt to run the changeset, which may fail. A best practice to effectively navigate this is to also include a rollback with your changeset:

```
<changeSet id="example-1" author="user">
  <createTable tableName="my_table">
    <column name="id" type="int"/>
  </createTable>
  <!-- Custom rollback logic -->
  <rollback>
    <dropTable tableName="my_table"/>
  </rollback>
</changeSet>
```

With this approach you can execute the following command:

```
>_ liquibase update-testing-rollback --defaults-
file=branchdev_liquibase.properties
```

This will apply the change, roll it back, then re-apply the change. **Note:** you will still need to make sure that the changes to the database aren't present before doing this.

Deployment order of operations:

1. Make changes to changelog files
2. Apply changes to individual dev database
3. Commit changelog to repo for PR and review
4. Apply changes to main dev database
5. Apply changes to production database